

Basic Python Datatypes

CS 101 Algorithmic Problem Solving

Fall 2024

There are a few basic datatypes that you will be working with quite frequently throughout the course, with more being introduced in the later stages of the semester. These basic datatypes are integers, floats, and string.

Integers

Integers are a numeric datatype that does not contain a decimal point. For example, 3 is an integer. If we assign a variable the value 3, we say the variable is an integer as the value of the variable is an integer.

```
n = 4
```

In this example, the variable n is an integer.

You might be tempted to use commas to represent large integers such as 1,000,000 to represent a million. However, Python does not interpret it as you'd expect; it instead treats it as a comma-separated sequence of 3 integers. We'll learn about this type of sequence later in the course but for now, remember that you should not use commas to represent large numbers.

Floating-Point Numbers

Floating-point numbers, or floats, are also a numeric datatype. Unlike strings however, they are characterized by the presence of a decimal point. For example, 3.142 is a float, and so is 4.0.

```
pi = 3.142
```

In this example, the variable pi is a float.

Both integers and floats can interact with each other in ways you would expect i.e. all mathematical operations are compatible with both integers and floats. However, it is important to note that certain mathematical operations will always result in a certain datatype as output. For example, the division operator (/) always returns a float; so 4/2 will return 2.0, 4.5/1.5 will return 3.0, 5/2 will return 2.5. Similarly, the floor division operator (//) always returns an integer; so 4//2 will return 2, 4.5/1.5 will return 3, 5//2 will return 2. You can always play around with this in Python to see what datatypes different operators return; however, for most intents and purposes, it is not necessary to memorize this.

Strings

Strings are a series of characters enclosed by quotation marks. These characters can include just about anything you'd want to represent.

```
intro = 'My name is Bond. James Bond.'
age = '34'
height = '184.7'
new_line = 'Hello!\nHow are you?'
tab_space = '\tI am fine.'
```

In the above example, we've created five string variables. As you can see, strings can contain alphabets, special characters, and numbers as long as they're enclosed between quotation marks. Note that you can use double or single quotation marks, but it is important to be consistent. The fourth and fifth variable we created contain a special type of character called an *escape character*. The `"\n"` character is used to represent a line break and the `"\t"` character is used to represent a tab space which is the equivalent of 4 spaces. If you were to print these two variables, you'd get the following output:

```
>>> print(new_line)
Hello!
How are you?
>>> print(tab_space)
    I am fine.
```

Note that while you can represent numbers in strings, `"34"` and `34` are not the same thing; one is a string and the other is an integer. It is important to keep this in mind when programming as strings cannot perform the same operations that integers and floats can perform. You will learn in detail about what operations strings can perform later on in the course.

One of these operations we want to introduce now is called string concatenation (don't worry, it's not as scary as it sounds). Suppose you have two strings, `"Hello!"` and `"How are you?"`. String concatenation is essentially adding these two strings together.

```
>>> print('Hello!' + 'How are you?')
Hello!How are you?
```

In the above example, two separate strings were joined together to make one string. As you can see, the syntax for concatenation is just the arithmetic plus sign.